**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА И ГОСУДАРСТВЕННОЙ СЛУЖБЫ при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»**

**СИБИРСКИЙ ИНСТИТУТ УПРАВЛЕНИЯ-филиал РАНХиГС**

Направление подготовки: «Информационные системы и программирование»

Образовательная программа: 09.02.07

# Проект

по дисциплине: Основы объектно-ориентированного программирования

на тему:

**«Разработка мобильного приложения карточной игры «Гвинт» на платформе Android»**

**Автор работы:**
Студент 4 курса группы К-ИСП-2302
Очной формы обучения
Вишняков Руслан Иванович

Новосибирск, 2026 г.

# СОДЕРЖАНИЕ

ВВЕДЕНИЕ ..... 3
1. Описание бизнес-логики проекта ..... 4
1.1 Назначение системы ..... 4
1.2 Основные сценарии использования ..... 5
1.3 Ключевые сущности и их поведение ..... 7
1.4 Бизнес-правила ..... 8
2. Тип архитектуры: монолит или микросервисы ..... 10
2.1 Обоснование выбора архитектуры ..... 10
2.2 Модули и слои ..... 10
2.3 Преимущества и недостатки выбранного подхода ..... 12
3. Хранение данных ..... 13
3.1 Способ хранения данных ..... 13
3.2 Хранимые сущности и атрибуты ..... 13
3.3 Схема данных ..... 14
3.4 Виды связей между сущностями ..... 15
4. Проверка безопасности приложений ..... 15
4.1 Актуальные угрозы ..... 15
4.2 Меры защиты ..... 16
4.3 Инкапсуляция и защита данных в проекте ..... 17
5. Законы и нормативные акты, необходимые для внедрения информационной системы . 18
5.1 Применимые нормативные акты ..... 18
5.2 Учёт требований при работе с данными ..... 19
5.3 Согласие пользователей и политика конфиденциальности ..... 20
6. Документы к каждому этапу процесса разработки ..... 21
6.1 Анализ требований ..... 21
6.2 Проектирование ..... 21
6.3 Разработка ..... 22
6.4 Тестирование ..... 22
6.5 Внедрение ..... 23
6.6 Сопровождение ..... 24
6.7 Заключение ..... 24
7. Индивидуальные акценты по ролям ..... 25
7.1 Роль: Инженер по качеству и тестированию (QA / Test automation) ..... 25
7.1.1 Подход к тестированию в проекте ..... 25
7.1.2 Объекты тестирования ..... 25

7.1.3 Виды проверок по ходу разработки ..... 26
7.1.4 Инструменты ..... 27
7.1.5 Пример выявленного и исправленного дефекта ..... 27
7.2 Роль: Технический писатель и DevOps ..... 28
7.2.1 Администрирование GitHub ..... 28
7.2.2 Структура репозитория ..... 28
7.2.3 README.md и документация ..... 29
7.2.4 UML-диаграммы и материалы для презентации ..... 29
7.2.5 Законы и нормативные документы ..... 30
7.3 Заключение ..... 30

# ВВЕДЕНИЕ

Целью проекта является разработка мобильного приложения «Гвинт», воспроизводящего оригинальную версию коллекционной карточной игры, с

возможностью сбора колоды, проведения сражений и использования карт с различными способностями в соответствии с оригиналом.

Задачи проекта:

1. Выполнить проектирование архитектуры мобильного приложения;
2. Реализовать основные игровые механики: размещение карт на трёх рядах, учёт погодных эффектов, применение способностей карт;
3. Разработать искусственный интеллект противника, принимающий решения на основе приоритетов ходов;
4. Создать пользовательский интерфейс для сборки колоды, проведения сражения, просмотра карт и управления игровым процессом;
5. Провести тестирование разработанного приложения, выявить и устранить ошибки.

# 1. Описание бизнес-логики проекта

## 1.1 Назначение системы

Проект представляет собой мобильное приложение — коллекционную карточную игру «Гвинт» для платформы Android. Система воспроизводит оригинальную версию игры, изначально появившуюся как мини-игра внутри компьютерной ролевой игры. Бизнес-логика — совокупность правил, алгоритмов и сценариев, определяющих поведение системы, — построена вокруг игрового процесса «Гвинта». Основная задача приложения — предоставить пользователю возможность играть в классический «Гвинт» на мобильном устройстве без обязательного подключения к интернету.

Актуальность проекта обусловлена тем, что современные коммерческие версии игры претерпели значительные изменения по сравнению с оригиналом: были переработаны правила, добавлены микротранзакции, изменены способности многих карт, введено обязательное сетевое подключение. Оригинальная версия осталась недоступной на мобильных платформах, что создаёт устойчивый спрос среди поклонников классической механики.

## 1.2 Основные сценарии использования

Сценарий использования — описание типового взаимодействия пользователя с системой, включающее последовательность действий и ожидаемый результат. Система поддерживает следующие сценарии:

— Сборка колоды — игрок открывает редактор колоды, добавляет и удаляет карты, формирует собственную стратегию.

— Сортировка карт по фракциям — игрок фильтрует коллекцию по принадлежности к фракции, что ускоряет поиск нужных карт.

— Выбор лидера — игрок выбирает карту лидера, обладающую уникальной способностью.

— Начало сражения — игрок запускает битву из главного меню с сохранённой колодой и лидером.

— Муллиган — в начале партии игрок заменяет до трёх карт в стартовой руке.

— Розыгрыш карты — игрок кликает по карте в руке, она анимировано перемещается на соответствующий ряд.

— Выбор ряда — для карт с нефиксированным расположением игрок выбирает ряд вручную.

— Использование способности карты — при выкладывании карты срабатывает её особая способность.

— Активация способности лидера — один раз за партию игрок активирует способность лидера.

— Пас — игрок отказывается от дальнейших ходов в текущем раунде.

— Просмотр карты долгое нажатие открывает диалог с увеличенным изображением и описанием способностей.

— Просмотр сброса игрок открывает свой сброс или сброс противника.

— Настройка звука регулировка громкости музыки и звуковых эффектов.

В формате пользовательских историй ключевые требования формулируются так: «Как игрок, я хочу собирать колоду из доступных карт, чтобы подготовить стратегию перед боем»; «Как игрок, я хочу активировать способность лидера один раз за партию, чтобы получить тактическое преимущество»; «Как игрок, я хочу заменить до трёх карт в стартовой руке, чтобы улучшить начальную руку».

Визуальную иллюстрацию вариантов использования можно увидеть на рисунке 1.

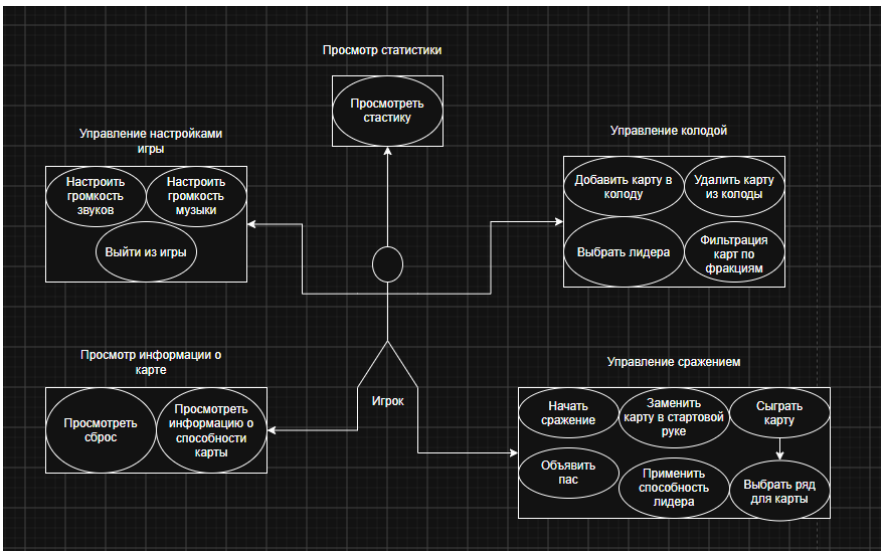



Рисунок 1. Диаграмма вариантов использования мобильного приложения «Гвинт»

## 1.3 Ключевые сущности и их поведение

Сущность — отдельный объект предметной области, обладающий набором атрибутов и определённым поведением. Бизнес-логика проекта построена вокруг следующих сущностей:

— Карта (Cards) — описывает игровую карту: изображение, название, силу, ряд, способность и фракцию.

— Состояние боя (BattleState) хранит данные о рядах обоих игроков, руках, колодах, сбросах, погодных эффектах и счёте.

— Экран сражения (BattleActivity) управляет ходом партии, обработкой действий игрока и работой искусственного интеллекта.

— Редактор колоды (DeckActivity) — отвечает за сборку колоды и выбор лидера.

— Главное меню (MainActivity) — точка входа в приложение.

— Справочник карт (CardsData) содержит данные обо всех картах, лидерах, группах связки и фракциях.

— Менеджер способностей (AbilityManager) обрабатывает применение способностей карт в зависимости от их типа.

— Калькулятор очков (ScoreCalculator) вычисляет силу карт и суммарный счёт с учётом погоды, горна и связки.

— Менеджер погоды (WeatherManager) управляет погодными эффектами и их визуальным отображением.

— Менеджер сброса (GraveyardManager) отвечает за перемещение карт в сброс и воскрешение.

— Искусственный интеллект (EnemyAI) принимает решения на основе системы приоритетов.

— Менеджер бонусов фракций (FactionAbilityManager) применяет способности фракций между раундами.

— Репозиторий колоды (DeckRepository) обеспечивает сохранение и загрузку колоды и лидера.

— Менеджер звука (SoundManager) — управляет фоновой музыкой и звуковыми эффектами.

— Менеджер сообщений (MessageManager) отображает всплывающие уведомления в ходе боя.

Каждая сущность отвечает строго за свою область и взаимодействует с другими через публичные методы, что обеспечивает контроль над изменениями состояния.

## 1.4 Бизнес-правила

Игровой процесс регулируется следующими правилами:

— Партия ведётся до двух побед в раундах.

— Раунд завершается, если оба игрока объявили пас или у одного из них закончились карты на руке.

— Право первого хода в раунде определяется случайным выбором, за исключением, если у игрока фракция «Скоя'таэли» — первым ходит игрок, если у противника — противник; если фракция «Скоя'таэли» есть у обоих — первый ход определяется случайным выбором.

— Победитель предыдущего раунда получает фракционный бонус (например, добор карты для Северных королевств).

— Карты-герои не подвержены воздействию погодных эффектов и способностей других карт.

— Карты погоды снижают силу обычных карт в соответствующих рядах до единицы: «Мороз» действует на ближний ряд, «Мгла» — на дальний, «Ливень» — на осадный.

— Карта «Ясное небо» снимает все активные погодные эффекты.

— Карта «Горн» удваивает силу обычных карт в ряду.

— Способность «Связка» умножает силу карт одной группы на их количество в ряду.

— Способность «Шпион» размещает карту на стороне противника, взамен позволяя игроку взять две карты из колоды.

— Способность «Медик» воскрешает одну карту из кладбища.

— Способность «Чучело» возвращает свою не-герой карту с поля в руку.

— Способность «Казнь» (execution) удаляет с поля карты противника с максимальной силой в указанных рядах (кроме карт-героев).

— Способность лидера доступна один раз за партию.

— В начале партии игрок может заменить до трёх карт в стартовой руке.

Все перечисленные правила формализуются в игровой логике, проверяются в ходе тестирования и фиксируются в сопроводительной документации.

## 2. Тип архитектуры: монолит или микросервисы

### 2.1 Обоснование выбора архитектуры

Архитектура программного обеспечения — это совокупность решений о структуре системы, определяющих её компоненты, их взаимодействие и правила организации. Существует два основных подхода: монолит, при котором приложение представляет собой единый развёртываемый модуль, и микросервисы, при котором система разделена на независимо развёртываемые сервисы, взаимодействующие по сети.

Для разрабатываемого проекта была выбрана монолитная архитектура. Это обосновано спецификой приложения: оно представляет собой автономное мобильное приложение для платформы Android, работающее без серверной части. Микросервисная архитектура предполагает разделение системы на независимые сервисы, взаимодействующие по сети, что для офлайн-игры на одном устройстве избыточно. Отсутствует сетевое взаимодействие, нет необходимости в масштабировании отдельных компонентов и в распределённых транзакциях.

### 2.2 Модули и слои

Внутри монолита выделены слои, обеспечивающие разделение ответственности — принцип, при котором каждая часть системы отвечает строго за свою область и не вмешивается в работу других.

Слой пользовательского интерфейса отвечает за отображение игрового процесса и обработку действий пользователя. В него входят экраны главного меню (MainActivity), выбора режима (ModeSelectActivity), редактора колоды (DeckActivity) и сражения (BattleActivity), а также диалог предпросмотра карт (CardPreviewDialogFragment) и адаптер для их отображения (CardsAdapter). За реализацию этого слоя отвечает разработчик интерфейса.

Слой игровой логики содержит правила и алгоритмы игры. В него входят состояние боя (BattleState), подсчёт очков (ScoreCalculator), управление погодными эффектами (WeatherManager), кладбищами (GraveyardManager),

способностями карт (AbilityManager), бонусами фракций (FactionAbilityManager) и искусственный интеллект (EnemyAI). Этот слой проектируется и реализуется архитектором ядра.

Слой данных отвечает за модель карты (Cards), справочник всех карт, лидеров и фракций (CardsData), а также за сохранение и загрузку колоды и лидера (DeckRepository). Дополнительно в этот слой входит менеджер статистики (StatsManager), сохраняющий счётчики побед, поражений и общего числа матчей. За корректность работы этого слоя отвечает специалист по безопасности и валидации данных.

Слой утилит содержит вспомогательные компоненты: менеджер звука (SoundManager), менеджер сообщений (MessageManager) и функции-расширения для работы со списками карт (CardExtensions).

Взаимодействие слоёв построено по принципу «сверху вниз»: пользовательский интерфейс вызывает методы игровой логики, игровая логика обращается к данным, а утилиты используются всеми слоями по мере необходимости. Такая организация исключает циклические зависимости и упрощает поддержку кода.

Схема взаимодействия компонентов представлена на рисунке 2.

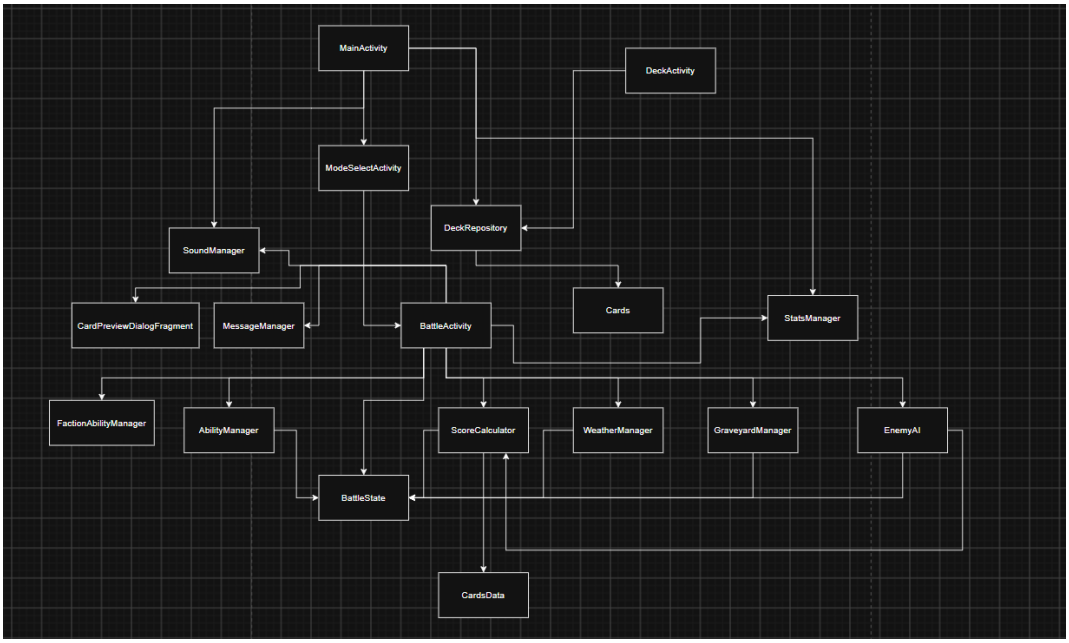



Рисунок 2. Диаграмма взаимодействия компонентов мобильного приложения «Гвинт»

## 2.3 Преимущества и недостатки выбранного подхода

Преимущества монолитной архитектуры для проекта:

— Простота развёртывания — приложение собирается в один APK-файл и устанавливается на устройство как единое целое.

— Отсутствие сетевых задержек — все вызовы между слоями происходят внутри одного процесса, что обеспечивает мгновенный отклик.

— Лёгкость отладки — все компоненты работают в едином процессе, что упрощает поиск ошибок.

— Отсутствие серверной инфраструктуры — приложение не требует хостинга, баз данных на сервере и сетевого взаимодействия.

— Недостатки монолитной архитектуры:

— Отсутствие жёсткой изоляции модулей — изменения в одном слое могут косвенно затронуть другой.

— Потенциальные сложности при масштабировании — если проект значительно вырастет, поддержка монолита усложнится.

— Единый артефакт — обновление любой части требует пересборки и повторной установки всего приложения.

Для текущих задач и планируемого расширения функциональности монолитный подход является оптимальным выбором. Он обеспечивает быструю разработку, простоту поддержки и отсутствие избыточной инфраструктуры.

## 3. Хранение данных

### 3.1 Способ хранения данных

Хранение данных — это способ организации долговременного сохранения информации, позволяющий приложению восстанавливать состояние между запусками. В мобильной разработке применяются реляционные базы данных, файловые хранилища, встроенные механизмы платформы и облачные сервисы.

В разрабатываемом проекте используется локальное хранение данных. Приложение работает автономно и не передаёт данные на серверы, поэтому необходимости в СУБД или облачных решениях нет. Для хранения применяются два механизма платформы Android:

SharedPreferences — встроенное хранилище типа «ключ-значение», предназначенное для небольших наборов параметров. В проекте в нём сохраняются настройки громкости музыки и звуковых эффектов. Третья группа SharedPreferences — «stats» — используется классом StatsManager для хранения агрегированной статистики матчей. Хранятся три целочисленных счётчика: games (всего матчей), wins (побед), losses (поражений). Запись выполняется методами addWin() и addLoss(), чтение — getGames(), getWins(), getLosses(). Эти данные используются в главном меню (MainActivity.updateStatsText()) и обновляются по завершении партии (BattleActivity.showEndGameScreen()).

JSON-сериализация — формат представления структурированных данных в виде текста. Используется для сохранения списка карт колоды и выбранного лидера. Сериализация выполняется с помощью библиотеки Gson, преобразующей объекты в JSON-строку и обратно. Полученная строка сохраняется в том же SharedPreferences.

Дополнительно используются ресурсы приложения для хранения статических данных: описания способностей карт в текстовом файле, а также исходный код в качестве справочника всех карт, лидеров и групп связки.

### 3.2 Хранимые сущности и атрибуты

Перечень хранимых сущностей и их атрибутов:

Карта — основная игровая единица. Атрибуты: изображение, название, сила, ряд, способность, видимость в сбросе, фракция.

Лидер — особая карта с уникальной способностью. Атрибуты: изображение, название, способность, фракция.

Колода — список карт, выбранных игроком для сражения.

Настройки — пользовательские параметры. Атрибуты: громкость музыки, громкость звуковых эффектов.

Статистика — агрегированные счётчики матчей. Атрибуты: games, wins, losses. Хранится в SharedPreferences «stats», доступ через StatsManager.

Описания способностей — текстовые пояснения к каждой способности карт.

### 3.3 Схема данных

Логическая схема хранимых данных, отражающая сущности и связи между ними, приведена на рисунке 3. Физически данные размещены в трёх группах SharedPreferences: «deck» (колода и лидер в JSON), «settings» (громкость музыки и звуков), «stats» (счётчики игр/побед/поражений).

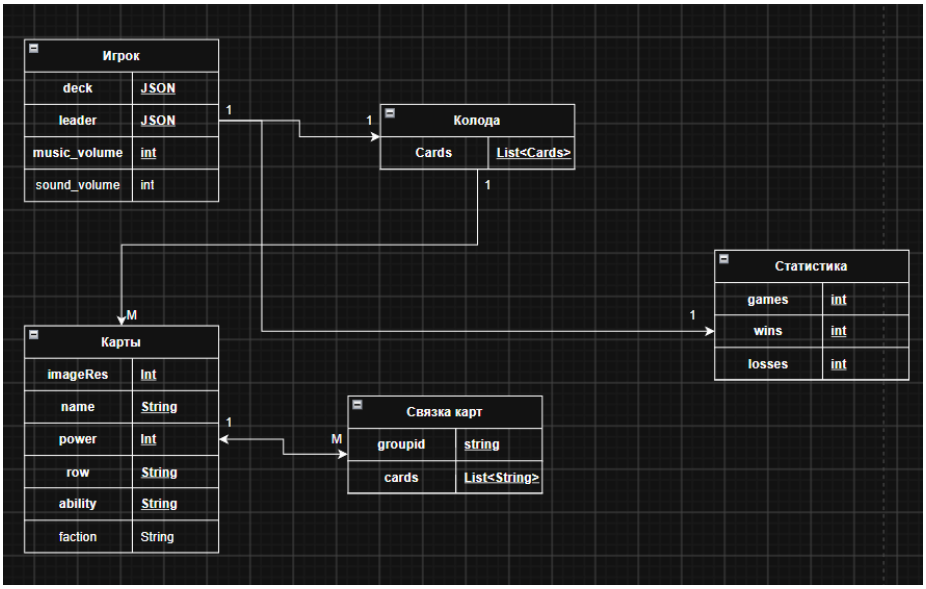



Рисунок 3. Схема данных мобильного приложения «Гвинт»

### 3.4 Виды связей между сущностями

Поскольку используется файловое хранилище, классические реляционные связи не применяются напрямую. Однако в качестве примера можно выделить три типа связей из данного проекта:

Связь «один-к-одному» — это отношение, при котором одной записи в первой сущности соответствует ровно одна запись во второй. В проекте такая связь проявляется в отношении игрока к выбранному лидеру: у игрока может быть только один лидер.

Связь «один-ко-многим» — это отношение, при котором одной записи в первой сущности соответствует несколько записей во второй. В проекте такая связь проявляется в отношении колоды к картам: одна колода содержит множество карт.

Связь «многие-ко-многим» — это отношение, при котором нескольким записям одной сущности соответствует несколько записей другой. В текущей версии проекта такая связь в явном виде не используется: справочники bondGroups и musterGroups реализованы как Map<String, List<String>>, где каждая карта однозначно определяется по имени и относится ровно к одной группе (getBondGroup / getMusterGroup возвращают единственное значение или null). Поэтому отношение «карта — группа» фактически является связью «один-ко-многим»: одна группа содержит несколько карт, но каждая карта входит только в одну группу. Связь M:N в проекте не применяется и приведена в тексте как теоретический случай для полноты классификации.

## 4. Проверка безопасности приложений

### 4.1 Актуальные угрозы

Информационная безопасность приложения — это состояние защищённости данных и логики системы от несанкционированного доступа, случайного повреждения и вредоносного воздействия. При анализе угроз важно учитывать специфику конкретного приложения, а не рассматривать универсальный список.

Для разрабатываемого проекта характерны следующие особенности:

— приложение работает локально, без сетевого взаимодействия;

— не использует базы данных SQL;

— не имеет веб-интерфейса;

— не собирает персональные данные пользователей.

Исходя из этого, часть типовых угроз неактуальна для проекта.

SQL-инъекции — это атаки, при которых в SQL-запрос внедряется вредоносный код. Не актуально, поскольку SQL не используется.

XSS — это межсайтовый скриптинг, при котором в веб-страницу внедряется вредоносный JavaScript. Не актуально, поскольку веб-интерфейса нет.

CSRF — это подделка межсайтовых запросов, при которой злоумышленник заставляет пользователя выполнить нежелательное действие. Не актуально, поскольку сетевые запросы отсутствуют.

Слабая аутентификация — не актуальна, поскольку учётные записи в приложении отсутствуют.

Актуальными для проекта остаются следующие три угрозы:

1. утечка данных при потере устройства — колода и настройки хранятся локально и доступны при физическом доступе к устройству;

2. повреждение сохранений — случайное или намеренное редактирование файлов хранилища может нарушить целостность данных;

3. некорректные данные в справочнике карт — ошибки в описаниях карт и способностей могут нарушить игровую логику.

## 4.2 Меры защиты

Для перечисленных угроз применяются следующие меры защиты.

Валидация данных — проверка корректности входных значений перед их использованием. При загрузке сохранений реализована проверка наличия ключа в SharedPreferences: если колода или лидер ранее не сохранялись, используются

значения по умолчанию, что исключает передачу пустых или неинициализированных данных в игровую логику.

Ограничение диапазона значений — числовые параметры, такие как громкость музыки и звука, ограничиваются допустимым диапазоном от 0 до 100. Это исключает выход за пределы корректных значений.

Обработка исключений — критичные операции чтения файлов и ресурсов оборачиваются в конструкцию try-catch, предотвращающую аварийное завершение приложения при ошибках.

Доступ к хранилищу SharedPreferences инкапсулирован в отдельном классе-репозитории, что исключает прямое взаимодействие других слоёв приложения с хранилищем.

Ограничение игровых действий — количество замен карт в стартовой руке ограничено тремя, что соответствует правилам игры и исключает возможность некорректного использования механики муллигана.

Хеширование паролей — не применяется, поскольку в приложении отсутствует аутентификация.

HTTPS — не применяется, поскольку приложение не взаимодействует с сетью.

Роли и права доступа — не применяются, поскольку приложение предназначено для одного пользователя.

Логирование — в проекте применяется ограниченно: события записываются средствами отладки в процессе разработки, но постоянное логирование в релизной версии не ведётся.

### 4.3 Инкапсуляция и защита данных в проекте

Инкапсуляция — принцип объектно-ориентированного программирования, при котором внутренние данные объекта скрыты от внешнего доступа и изменяются только через публичные методы.

В проекте реализованы следующие механизмы инкапсуляции и защиты данных.

Приватные атрибуты. Большинство полей состояния и менеджеров объявлены как приватные и недоступны для изменения извне. Это исключает случайное или намеренное изменение состояния в обход методов класса.

Публичные методы как контролируемые точки входа. Изменение состояния боя происходит только через публичные методы менеджеров: применение способностей, розыгрыш карт, смена хода. Это позволяет централизованно контролировать все изменения и отслеживать их источник.

Модель карты как неизменяемая структура. Большая часть атрибутов карты — такие как изображение, название, ряд, способность, фракция — объявлены как неизменяемые. Изменяемыми остаются только те поля, которые действительно изменяются в ходе игры: сила карты и признак видимости в сбросе.

Проверки типов и значений. Все числовые параметры, получаемые из внешних источников, приводятся к допустимому диапазону. Строковые параметры проверяются на соответствие ожидаемым значениям.

Изоляция хранилища. Прямой доступ к SharedPreferences осуществляется только из класса-репозитория. Остальные слои приложения работают с сохранёнными данными через методы репозитория, что исключает случайное повреждение данных.

Такая организация обеспечивает устойчивость приложения к некорректным данным и снижает риск случайного нарушения игровой логики.

## 5. Законы и нормативные акты, необходимые для внедрения информационной системы

### 5.1 Применимые нормативные акты

Информационная система — это совокупность программных и аппаратных средств, предназначенных для сбора, хранения, обработки и выдачи информации. Внедрение любой информационной системы регулируется законодательством в области информации и защиты данных. Для разрабатываемого проекта применимы следующие нормативные акты.

Федеральный закон № 152-ФЗ «О персональных данных» — регулирует отношения, связанные с обработкой персональных данных, то есть любых сведений, относящихся к конкретному человеку. Закон устанавливает требования к сбору, хранению, обработке и защите таких данных. Для разрабатываемого приложения требования этого закона минимальны, поскольку система не собирает и не обрабатывает персональные данные пользователей.

Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и защите информации» — устанавливает общие принципы работы с информацией, определяет правовой режим её распространения, а также регулирует отношения между участниками информационного обмена. Закон применим к проекту в общем виде как к любому программному продукту, работающему с информацией.

Регламент GDPR — общий регламент по защите данных, действующий на территории Европейского союза. Для проекта неактуален, поскольку приложение ориентировано на российский рынок и не распространяется среди пользователей ЕС.

Локальные политики компании — внутренние документы организации, регулирующие использование программного обеспечения. Применимы, если приложение будет использоваться в корпоративной среде.

### 5.2 Учёт требований при работе с данными

При сборе, хранении и обработке данных в проекте соблюдаются следующие принципы.

Минимизация данных — принцип, при котором собираются только те данные, которые необходимы для работы системы. Приложение хранит колоду игрока, пользовательские настройки громкости и агрегированную статистику матчей (всего игр, побед, поражений). Персональные сведения — имя, контакты, местоположение — не собираются.

Локальное хранение — все данные сохраняются исключительно на устройстве пользователя и не передаются на сторонние серверы. Это исключает необходимость выполнения требований к защите данных при передаче по сети.

Отсутствие трансграничной передачи — данные не покидают устройство, что снимает вопросы о соответствии требованиям законодательства других стран.

Прозрачность хранения — пользователь может самостоятельно удалить все данные, удалив приложение. Политика конфиденциальности при этом не требуется, поскольку персональные данные не обрабатываются.

### 5.3 Согласие пользователей и политика конфиденциальности

Согласие пользователя — это добровольное разрешение на обработку персональных данных. Поскольку приложение не собирает персональные данные, получение согласия не требуется.

Политика конфиденциальности — документ, описывающий, какие данные собирает приложение и как они используются. В текущей версии проекта политика конфиденциальности не разрабатывается, так как приложение не собирает и не передаёт данные.

Однако при публикации приложения в магазинах Google Play или RuStore потребуется соблюдение правил этих площадок. Правила требуют предоставления декларации о работе с данными даже для приложений, которые их не собирают. Поэтому перед возможной публикацией потребуется подготовить декларацию, подтверждающую отсутствие сбора персональных данных.

## 6. Документы к каждому этапу процесса разработки

### 6.1 Анализ требований

Анализ требований — это этап разработки, на котором формулируются цели системы, её функции и ограничения. От качества этого этапа зависит, насколько точно конечный продукт будет соответствовать ожиданиям.

На этом этапе формируются следующие документы.

Техническое задание — основной документ, описывающий назначение системы, требования к функциональности, интерфейсу и качеству. Определяет, что именно должна делать система и в каких условиях работать.

Список требований — перечень конкретных функциональных и нефункциональных требований, представленных в виде пронумерованных пунктов. Используется для контроля полноты реализации.

Пользовательские истории — краткие описания сценариев использования системы от лица пользователя. Формулируются в виде «как пользователь, я хочу выполнить действие, чтобы получить результат». Применяются для приоритизации разработки.

После утверждения требований проект переходит к этапу проектирования.

### 6.2 Проектирование

Проектирование — это этап, на котором определяются структура системы, её компоненты и их взаимодействие. На основе требований формируется архитектурное решение и детальная модель будущего приложения.

На этом этапе создаются следующие документы.

UML-диаграмма классов — схема, отображающая классы системы и связи между ними. Показывает, из каких сущностей состоит приложение и как они взаимодействуют друг с другом.

UML-диаграмма последовательностей — схема, показывающая порядок вызовов методов между объектами при выполнении конкретного сценария. Используется для документирования сложных сценариев, например розыгрыша карты или применения способности.

ER-диаграмма — схема, описывающая структуру хранения данных и связи между сущностями. Применяется для документирования формата хранения.

Схема архитектуры — общее представление о слоях и модулях приложения, их назначении и взаимодействии.

После утверждения проектной документации начинается разработка.

## 6.3 Разработка

Разработка — это этап реализации системы в соответствии с проектной документацией. На этом этапе создаётся исходный код приложения и ведётся рабочая документация.

На этом этапе формируются следующие документы.

Описание модулей — документ, в котором перечислены модули приложения и указано их назначение. Включает краткое описание ответственности каждого модуля и его места в архитектуре.

Структура кода — описание организации исходного кода: какие папки и пакеты используются, какие файлы за что отвечают, как группируются классы.

README — основной файл документации проекта, размещаемый в корне репозитория. Содержит краткое описание приложения, инструкцию по сборке и запуску, примеры использования и перечень зависимостей.

API — программный интерфейс взаимодействия между модулями. В проекте представлен набором публичных методов классов игровой логики, через которые осуществляется доступ к игровому состоянию.

После завершения разработки функциональности начинается тестирование.

## 6.4 Тестирование

Тестирование — это этап проверки соответствия системы заявленным требованиям. Цель этапа — выявить и устранить дефекты до передачи приложения пользователям.

На этом этапе формируются следующие документы.

Тест-план — документ, описывающий, какие модули и методы подлежат тестированию, какие виды тестов применяются и какие критерии приёмки используются.

Чек-листы — структурированные списки проверок, выполняемых при тестировании. Используются для ручного тестирования и проверки типовых сценариев.

Отчёты о дефектах — документы, описывающие выявленные ошибки: условия воспроизведения, ожидаемое и фактическое поведение, критичность. Применяются для отслеживания процесса исправлений.

Автотесты — программные тесты, выполняемые без участия человека. Применяются для проверки игровой логики и предотвращения регрессий при доработках.

После успешного прохождения тестирования приложение переходит к внедрению.

### 6.5 Внедрение

Внедрение — это этап передачи системы пользователям и её развёртывания. На этом этапе готовится сопроводительная документация для конечного пользователя.

На этом этапе формируются следующие документы.

Инструкция по развёртыванию — документ, описывающий процесс установки приложения на устройство. Включает требования к среде, порядок сборки APK и способы установки.

Руководство пользователя — документ, описывающий основные сценарии работы с приложением. Содержит пошаговые инструкции по сборке колоды, началу сражения и настройке приложения.

План миграции — документ, описывающий перенос данных при обновлении версии. В текущей архитектуре формат сохранений (JSON в SharedPreferences) является стабильным и не требует миграции между версиями.

После выпуска приложения начинается этап сопровождения.

## 6.6 Сопровождение

Сопровождение — это этап поддержки системы после её выпуска. Включает выпуск обновлений, исправление ошибок и обеспечение сохранности данных пользователей.

На этом этапе формируются следующие документы.

Регламент обновлений — документ, описывающий порядок выпуска новых версий: какие изменения вносятся, как проводится тестирование, какие каналы распространения используются.

Резервное копирование — процедура сохранения данных для предотвращения их утраты. В проекте данные хранятся локально в SharedPreferences и могут быть включены в системное резервное копирование Android (Auto Backup) в соответствии с настройками платформы.

Мониторинг — процедура отслеживания состояния системы в процессе эксплуатации. В проекте используется класс DebugLogger для записи ключевых событий в процессе разработки, а также стандартный вывод стеков исключений. Внедрение внешних систем мониторинга (например, Crashlytics) предусмотрено на этапе промышленной эксплуатации.

## 6.7 Заключение

Все перечисленные документы ведутся в едином репозитории проекта. Это обеспечивает централизованное хранение информации, удобный доступ к ней для всех участников команды и возможность отслеживания изменений. Документация обновляется по мере развития проекта и служит основой для передачи знаний между участниками и для дальнейшего сопровождения приложения.

# 7. Индивидуальные акценты по ролям

Данная глава посвящена индивидуальному вкладу автора в разработку проекта. Автор совмещает две роли: инженера по качеству и тестированию (QA) и технического писателя (DevOps). Основной ролью является тестирование, поэтому ей уделено центральное место.

## 7.1 Роль: Инженер по качеству и тестированию (QA / Test automation)

### 7.1.1 Подход к тестированию в проекте

Поскольку приложение находится в активной разработке (пул карт и способностей расширяется, интерфейс дорабатывается), тестирование не выделяется в отдельный финальный этап. Проверки выполняются по мере готовности отдельных модулей: сначала проверяется базовая логика, затем — взаимодействие компонентов, а при добавлении новых карт или механик проводится регрессионная проверка ранее работавших сценариев, чтобы убедиться, что изменения не сломали существующее поведение. Такой подход позволяет выявлять дефекты на ранних стадиях и не накапливать их к концу разработки.

### 7.1.2 Объекты тестирования

Тестирование охватывает как игровое ядро, так и слой взаимодействия с пользователем. Основными объектами выступают:

- ScoreCalculator — формулы подсчёта очков, включая взаимодействие «Горна», «Связки», погодных эффектов и «Морального духа».
- AbilityManager — срабатывание способностей (Шпион, Медик, Чучело, погодные карты) и корректность их применения к разным типам карт.
- EnemyAI — адекватность решений бота: выбор хода, своевременный пас, приоритизация карт.
- DeckRepository — сохранение, загрузка и целостность колоды и выбранного лидера.

- BattleState и менеджеры (WeatherManager, GraveyardManager, FactionAbilityManager) — корректность изменения состояния боя при розыгрыше карт, смене раундов и активации способностей.
- Слой UI (BattleActivity, DeckActivity, MainActivity) — корректность передачи действий пользователя в ядро и перерисовки экрана после ответа ядра.

### 7.1.3 Виды проверок по ходу разработки

В зависимости от этапа готовности функциональности применяются разные виды проверок:

- Модульные проверки (Unit Testing) — выполняются по мере написания отдельных методов. Проверяют изолированную логику: расчёт силы карты с учётом погоды, подсчёт очков ряда, работу функций-расширений (например, withoutSpies()). Такие проверки удобны тем, что не требуют запуска всего приложения.
- Интеграционные проверки — выполняются, когда модуль готов к взаимодействию с другими. Проверяют цепочку «действие пользователя → вызов метода ядра → обновление состояния → колбэк в UI». Например, розыгрыш карты через playCard() и последующая перерисовка поля.
- Проверка граничных условий — выполняется при готовности соответствующих сценариев: пустая рука, отсутствие карт в колоде, достижение лимита муллигана (3 карты), повторная активация способности лидера, розыгрыш карты при заполненном ряду. Эти проверки особенно важны, так как именно на границах чаще всего возникают ошибки.
- Проверка невалидных данных — выполняется при работе с сохранениями и внешними данными. Например, подача повреждённого JSON в DeckRepository для проверки fallback-механизмов (возврат к значению по умолчанию).

- Проверка полиморфного поведения — выполняется после стабилизации интерфейса BattleInterface. Проверяется, что AbilityManager корректно вызывает методы activateFrost(), addCardToRow() и другие через ссылку на интерфейс, не зная конкретной реализации BattleActivity.

Полный перечень проверок, шагов, ожидаемых результатов и мест для скриншотов приведён в отдельном документе. На момент написания, в нём описано 15 тестов, покрывающих ключевые сценарии: запуск приложения, работу с колодой, выбор лидера и режима, муллиган, розыгрыш карт, активацию способности лидера, пас, погодные эффекты, просмотр карт и завершение партии со статистикой. Результаты выполнения приложены к каждому тесту.

### 7.1.4 Инструменты

Для написания автотестов используется стандартный фреймворк JUnit 4 (встроен в Android Studio). Для UI-тестирования (проверки нажатий и переходов между экранами) применяется Espresso. Так как проект на Kotlin, используются его платформенные аналоги). В текущей версии проекта автотесты ограничены стандартными проверками, сгенерированными Android Studio (ExampleUnitTest и ExampleInstrumentedTest).

### 7.1.5 Пример выявленного и исправленного дефекта

В ходе проверки ScoreCalculator был выявлен дефект: неверный порядок применения модификаторов силы. При одновременном действии «Горна» (x2), «Морального духа» (+1) и «Связки» карта считалась неправильно (описание дефекта). Ожидаемое поведение: сначала применяется «Связка», затем «Моральный дух», затем «Горн». Фактическое: модификаторы применялись в обратном порядке. Дефект исправлен, сценарий повторно проверен и пройден успешно.

По мере дальнейшей разработки тесты будут дополняться и систематизироваться, а по завершении работы над проектом все тестовые

сценарии и примеры проверок будут собраны в отдельный файл тестов, прилагаемый к репозиторию и документации.

### 7.2 Роль: Технический писатель и DevOps

#### 7.2.1 Администрирование GitHub

Весь исходный код проекта хранится в публичном репозитории GitHub. Ведение разработки осуществляется через отдельные ветки для каждой новой функции или исправления (feature branches). Слияние изменений в основную ветку main происходит через механизм merge requests (Pull Requests), что позволяет проводить код-ревью и отслеживать историю изменений.

#### 7.2.2 Структура репозитория

Проект организован по слоям (пакетам):

- data — модели (Cards) и репозиторий (DeckRepository).
- domain — игровая логика (ScoreCalculator, EnemyAI, AbilityManager).
- ui — экраны приложения (MainActivity, BattleActivity).
- utils — вспомогательные классы (SoundManager, MessageManager).

Представлен на рисунке 4.

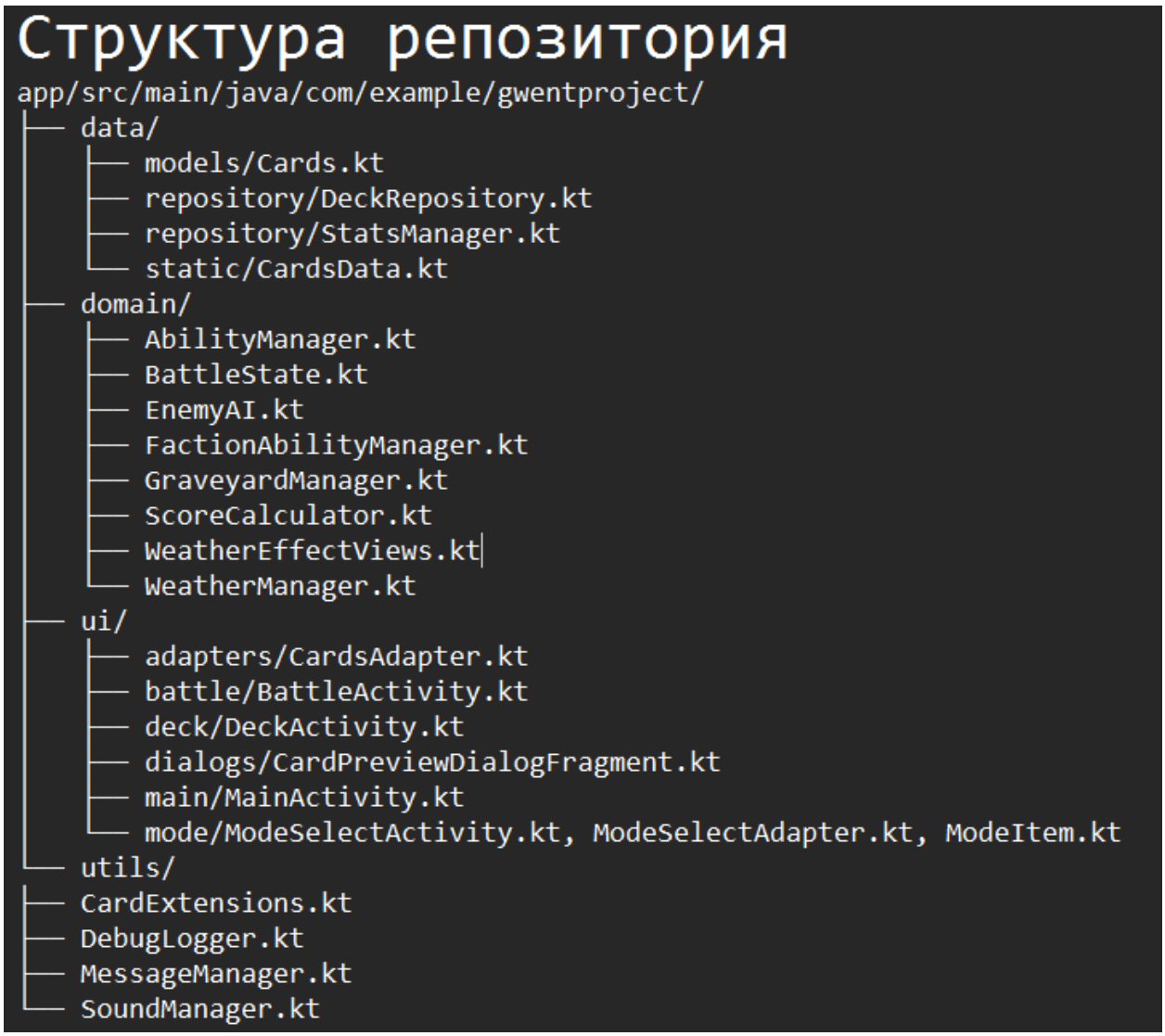



Рисунок 4. Структура репозитория

### 7.2.3 README.md и документация

В корне репозитория расположен файл README.md, содержащий описание проекта и инструкцию по сборке. Для ключевых классов игровой логики (AbilityManager, ScoreCalculator, EnemyAI) написаны комментарии (Kdoc), объясняющие назначение методов и параметров. Это упрощает вхождение в проект новым разработчикам.

### 7.2.4 UML-диаграммы и материалы для презентации

Совместно с архитектором ядра подготовлена UML-диаграмма классов (отображающая иерархию и композицию) и диаграмма компонентов. Все схемы

сохранены в репозитории в формате. drawio для возможности их дальнейшего редактирования. Также были собраны материалы для финальной защиты проекта.

### 7.2.5 Законы и нормативные документы

Совместно со специалистом по безопасности проведён анализ нормативных актов (152-ФЗ, 149-ФЗ). В документацию добавлен раздел, обосновывающий отсутствие необходимости в политике конфиденциальности (так как персональные данные не собираются), а также указано требование о подготовке декларации для магазинов приложений.

## 7.3 Заключение

В ходе работы автор выполнил роль инженера по тестированию (проверив игровую логику на граничные случаи и исправив дефекты) и технического писателя (оформив репозиторий, документацию и UML-схемы). Совмещение этих ролей обеспечило высокое качество кода и прозрачность проекта для проверки.